

Algoritmi e Strutture Dati – Parte A – 12/06/2020

Esercizio A1, punti ≥ 8

Trovare i limiti superiori e inferiori più stretti possibili per la seguente equazione di ricorrenza, utilizzando il metodo di sostituzione (detto anche per tentativi):

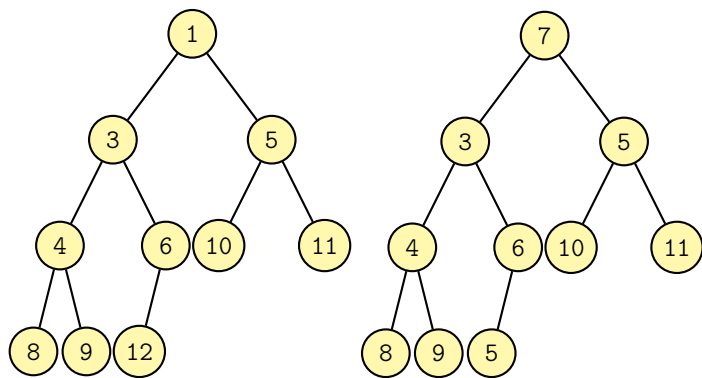
$$T(n) = \begin{cases} T(\lfloor \frac{n}{2} \rfloor) + 2T(\lfloor \frac{n}{2\sqrt{2}} \rfloor) + 7T(\lfloor \frac{n}{4} \rfloor) + n^2 & n \geq 4 \\ 1 & n < 4 \end{cases}$$

Esercizio A2, punti ≥ 10

Scrivere un algoritmo **boolean** `isMinHeap(TREE T)` che prenda in input un albero binario completo composto da n nodi contenenti valori interi nel campo *value* e restituisca **true** se l'albero rispetta le regole dei Min Heap, **false** altrimenti.

Discutere correttezza e complessità computazionale dell'algoritmo proposto.

Nella figura seguente, l'albero di sinistra è un albero completo che rispetta le regole di minheap, l'albero di destra non le rispetta.



Esercizio A3 – Mystery array – Punti ≥ 12

Un *mystery array* è una struttura dati di tipo sequenza, che permette di accedere ai suoi elementi tramite indici. Tuttavia, non ha primitive per ottenere il numero di elementi in esso contenuti. Se si tenta di leggere un indice i presente in un mystery array A , viene restituito il valore $A[i]$ corrispondente. Se i è superiore alla dimensione del vettore, viene restituito il valore speciale $\perp$.

Esempio: Sia $A = [-5, -3, 11, 13, 11]$ un mystery array di 5 elementi. $A[4]$ restituisce 13, mentre $A[1000]$ restituisce $\perp$.

Scrivere un algoritmo

int countNegatives(MISTERYARRAY A)

che prenda in input un mystery array non vuoto, ordinato A contenente interi, e restituisca il numero di valori negativi presenti nel mystery array.

Spiegare il funzionamento e discutere la complessità computazionale dell'algoritmo proposto. Algoritmi di complessità $\Theta(n)$ o superiore, dove n è l'effettiva dimensione del mystery array, non verranno presi in considerazione.